

IBM RESEARCH TECHNICAL REPORT

MELLEA SKILLS COMPILER: Compiling and Certifying Agent Skills with MELLEA

Elizabeth M. Daly, Dhaval Salwala, Inge Vejsbjerg, Seshu Tirupathi, Rebecka Nordenl w,
Lamogha Chiazor, Jessica He, Kush R. Varshney, and Jordan McAfoose
IBM Research

May 2026

Abstract

AI agents increasingly ship as natural language specifications: Markdown files, YAML configurations, and system prompts, executed by large language models without formal verification, runtime monitoring, or compliance documentation. This report presents the MELLEA SKILLS COMPILER pipeline, a sequence of steps that transforms agent skill specifications into typed, instrumented programs with policy-driven guardrails and auditable execution traces. The skills are compiled into programs written in MELLEA, a language for generative computing. The pipeline composes three IBM Research technologies: (1) MELLEA-FY, a specification compiler that decomposes natural language agent skill specifications into typed generative programs; (2) Granite Guardian, a risk detection model integrated via MELLEA’s hook system for per-generation safety assessment; and (3) AI Atlas Nexus, a governance knowledge graph that maps use-case descriptions to applicable risks across multiple taxonomies.

The compiler component, MELLEA-FY, structures compilation as a multi-step workflow with narrowly-scoped LLM invocations, an introspected MELLEA API reference that grounds code generation, and a static-validation pass with a bounded repair loop. Compile itself currently runs on a frontier-class model; the compiled pipeline is intended to execute reliably on small to mid-size task models (5B–24B), and the architecture draws on patterns from the compiler community (error-recovering parsers, version-predicated migration recipes, structured emitters) and from research on grounded, verified code generation, both to make the compiler robust today and to open the door to running some compile-time steps on smaller models in future work. This report describes the architecture and design rationale; an empirical evaluation is in progress and will be reported separately.



<https://github.com/generative-computing/mellea-skills-compiler>

1. Introduction

Agentic AI saw rapid and globally broad adoption in early 2026, driven primarily by OpenClaw, which Nvidia CEO Jensen Huang called “definitely the next ChatGPT” [Likos, 2026]. The numbers

back up the pronouncement: a 925% increase in traffic from February to March 2026, and over 300,000 stars on GitHub in less than four months [Hillebrandt, 2026]. The key to this movement is registries of agent skills written as markdown files. Skills are consumed by OpenClaw and by proprietary offerings like Claude Cowork, and unlike conventional chatbots that only respond to user input, OpenClaw agents can take actions autonomously on the user’s behalf.¹

Alongside the rapid growth of the claw ecosystem, there has been a so-called ClawHavoc. In February 2026, numerous security researchers found that double-digit percentages of skills on OpenClaw’s official registry, ClawHub, were malicious [Alex and Yomtov, 2026, Holzbauer et al., 2026]. The pattern extends beyond the registry: a Snyk audit reported that 36% of the code that provides AI agents their functions contains at least one notable security flaw, and an unsecured database belonging to Moltbook, the first social network for AI agents themselves, exposed roughly 1.5 million API keys; the same setting also enables prompt-injection attacks that can alter agent behaviour through ordinary plain-text posts [Smith, 2026]. Moreover, there are concerns that agentic systems such as these may be prone to action-completion bias, potentially introducing instability into workflows. Scholars have argued that such agents risk lacking the discernment, discretion, and metacognitive wisdom needed to balance autonomous action-taking with appropriate restraint [Johnson et al., 2025]. In addition, there are concerns about the cost of running agentic systems based on very large frontier-class language models; recently, Anthropic disallowed LLM inference calls originating in third-party agents from being counted in flat-rate pricing plans because the workload patterns of such calls are not tenable [Constantin, 2026].

We can imagine two approaches for bringing order and governance to claws, and to agent skills more generally. The extrinsic approach uses sandboxing and policy-based external guardrails to block unauthorised actions before they execute; NVIDIA’s NemoClaw is one example. The intrinsic approach brings discernment and patterned thought within the skill package itself. External guardrails alone may be insufficient where skills are specified in natural language, since ambiguous or unforeseen interpretations can escape policy enforcement, and manual configuration burden can lead users to bypass safe deployment practices altogether.² In this report, we contribute to the intrinsic approach through the MELLEA SKILLS COMPILER pipeline, which transforms skill specifications (Markdown files and the equivalents written for runtimes such as CrewAI, LangGraph, and Letta) into validated, structured generative programs with embedded guardrails.

The ecosystem is at an inflection point. Skills are being deployed across model tiers by users who did not write them, on inputs the original authors did not anticipate. The specification format is sound, but specifications alone do not guarantee reliable execution at scale. Skills developed for frontier models may not work on the smaller models that cost-sensitive deployment requires. Compiling skills into structured pipelines aims to let smaller language models perform with the capability of larger ones, and addresses the cost issue at the same time. The compiled packages can be shared on registries in as open and portable a way as current skills.

A note on terminology: in our usage, a compiled skill package is not an agent or an agentic framework, but a typed, validated, governed program that any MELLEA-compatible runtime can execute.

The proposed pipeline brings together our previous work and that of our colleagues at IBM

¹The term ‘claw’ has become a loose but pervasive label across the ecosystem. At the skill level the naming is consistent (the official registry is ClawHub, and individual skills use claw-based terminology), but at the agent level it varies: some derivative products adopt ‘claw’ directly (NanoClaw, NullClaw, ZeroClaw), others retain the Clawdbot branding, and others use entirely independent names. As a result, ‘claw’ refers to the ecosystem broadly rather than to any single technical component.

²https://www.linkedin.com/posts/linnana_aiagents-enterpriseai-buildingwithai-activity-7440045620049915906-D6kW

Research on: MELLEA (a language and system that decomposes complex tasks involving generative AI into structured sub-steps that are individually executed, validated against formal requirements, and automatically repaired if they fail [IBM Research, 2025]), AI Atlas Nexus (a knowledge graph and ontology navigation tool that consolidates AI risk and capability taxonomies [Vejsbjerg et al., 2026]), and Granite Guardian (a guardrail model that detects agentic harms in context, prompts, and responses [Padhi et al., 2025]). The primary component of the pipeline is a novel MELLEA-FY skill.

The compilation stage is a *structured ten-step workflow* that bounds and scopes LLM invocations, with an introspected MELLEA API reference injected as grounding context, deterministic Python emitters for structured commitments, an explicit dependency-disposition step, and a static-validation pass feeding a bounded repair loop. Compile itself runs on a frontier-class model today; the design target is that the *compiled pipeline* executes reliably on small to mid-size task models. The architecture draws on patterns from the compiler community (error-recovering parsers, version-predicated migration recipes, structured emitters) [Marsh and Astral, 2024, Moderne, Inc., 2024] and on research on grounded, verified code generation [Zhou et al., 2023b, Xia et al., 2024, Jiang et al., 2024], both to make the compiler robust today and to open the door to running some compile-time steps on smaller models in future work.

This report contributes:

1. **An architecture.** A compile/instrument/certify pipeline, with an experimental export stage, that transforms a natural-language agent skill specification into a typed, governed, instrumented MELLEA package and re-targets it to alternative deployment harnesses (MCP, LangGraph, Claude Code) without re-implementation. The compile stage is described in full at the level of its ten-step workflow, including a five-axis classification (with source-runtime detection across Markdown, Claude Code, OpenClaw, CrewAI, LangGraph, Letta, AutoGen, OpenAI Agents SDK, and smolagents) and an eight-disposition dependency audit.
2. **A design rationale.** Our work draws on prior work and best practices in compiler construction and code generation; the supporting evidence is laid out in §2 and §3.1.
3. **Outcome variables for evaluation.** The architecture makes several new outcomes empirically measurable; the evaluation itself is underway and will be reported separately.

This report focuses on the architecture, the design rationale, and the planned evaluation; the empirical evaluation is in progress and will be reported in a follow-up paper. The compiler is released as open source at <https://github.com/generative-computing/mellea-skills-compiler>.

2. Background and Related Work

In this section, we briefly overview the rapidly-evolving world of agentic AI skills, ecosystems, security and governance.

2.1 OpenClaw Derivative Ecosystem

The OpenClaw derivative ecosystem marks a shift in how autonomous agents are deployed in sociotechnical systems. Over the last several weeks, this landscape has expanded to include over forty projects that move beyond the original codebase to address gaps in security, scale, and specialization. This movement builds upon the foundational multi-agent orchestration patterns established by frameworks like AutoGen [Wu et al., 2024] and the role-based collaborative architectures of MetaGPT [Hong et al., 2023].

We see a clear stratification where enterprise-grade wrappers like NVIDIA’s NemoClaw provide policy-based sandboxing and filesystem isolation, while the minimalist movement produces systems

like NullClaw, a static binary that achieves sub-two-millisecond boot times for industrial applications. These developments suggest that while the core logic of an autonomous agent may be increasingly commoditised, there is room for innovation in agentic governance as we are attempting through the MELLEA SKILLS COMPILER pipeline (but intrinsically rather than extrinsically as in other efforts).

At the architectural level, the community appears to be converging on a set of principled constraints that prioritize transparency and auditability over feature accumulation. This evolution is rooted in the tool-use capabilities suggested by Toolformer [Schick et al., 2023], which demonstrated that LLMs could learn to invoke functional APIs. Projects with the broadest adoption, such as NanoClaw and ZeroClaw, offer a readable, verifiable kernel that developers can fully comprehend in minutes. This shift toward technical rigor is complemented by a human-centric focus on domain-specific frameworks, like ScienceClaw, which replaces central orchestration with a plannerless coordination model based on scientific artifact lineage. Similarly, ClawBolt is a locally-hosted SMS-interfaced agentic framework specialized for the business operations of skilled trades like carpentry and plumbing.

2.2 Agent Security and Governance Frameworks

Robust agent governance frameworks have transitioned from high-level ethical desiderata to specific technical constraints. The NIST AI Risk Management Framework and its AI 600-1 companion generative AI profile provide a taxonomy involving three functions: map, measure, and manage. The map function is increasingly supported by tools [Greshake et al., 2023, Slattery et al., 2026] like the AI Atlas Nexus, which uses an open-source ontology to link abstract risks to specific datasets and mitigations [Vejsbjerg et al., 2026]. Tooling like guardian models and guardian agents bridge into the measure and manage functions [Padhi et al., 2025, Xiang et al., 2024, Tirupathi et al., 2025, Wang et al., 2025a]. They can quantitatively detect and prevent various unwanted agentic actions and hallucinations, but are not yet typically embedded within the agent’s functional logic, as we outline with the MELLEA SKILLS COMPILER pipeline.

At the infrastructure level, systems like NemoClaw address where an agent runs through container isolation and hardware deployment policies. However, infrastructure-level security is only one component of a trustworthy system. We must also consider whether an agent’s autonomous actions conform to established policies. While orchestration frameworks like LangChain provide tool-use governance through permission systems, they often lack the specification-level analysis and compliance documentation required for formal oversight required by modern regulatory standards like the EU AI Act. Such documentation and certification is a component of the compiled skill packages that the MELLEA SKILLS COMPILER pipeline produces.

2.3 Specification Compilation

Natural language agent specifications, markdown documents containing instructions, constraints, morals, examples, and domain knowledge, have become the dominant authoring format for AI agent skills. Anthropic’s skill framework defines a skill as a folder containing a `SKILL.md` file with optional supporting directories for scripts, references, and assets, loaded via a three-level progressive disclosure system [Anthropic, 2025]. Similar patterns exist across the industry: OpenAI’s GPT instructions, Google’s Gemini system prompts, and open-source agent frameworks all rely on natural language specifications as the primary interface between human intent and model behaviour.

These specifications are inherently designed for frontier-class models capable of resolving implicit contradictions across dozens of interacting rules. However, a significant mismatch arises as deployment economics increasingly favor small language models, ranging up to ~25B parameters. The industry

is currently witnessing a widespread but poorly characterized phenomenon where specifications authored for highly-capable frontier reasoning are executed by models with substantially reduced capability.

This paradigm assumes a level of model capability that is empirically validated on frontier systems but remains unaddressed for smaller architectures. Anthropic identifies three core categories of agent skills: (1) Document & Asset Creation, (2) Workflow Automation, and (3) MCP Enhancement, which align with the reasoning-archetype taxonomy that the MELLEA SKILLS COMPILER’s Step 0 classifier uses (Appendix A). For example, the Sentry code review skill serves as a canonical exemplar of Category 3 (MCP Enhancement). Whether structured decomposition can recover some of the output quality lost when small task models execute specifications designed for larger ones is an empirical question that the ongoing evaluation tests directly. One can also imagine small models that are especially tuned or co-designed for running structured decompositions.

The intuition that breaking complex tasks into simpler components improves performance is well-supported by techniques such as chain-of-thought prompting [Wei et al., 2022] and least-to-most prompting [Zhou et al., 2023a]. While these methods focus on reasoning decomposition (splitting a single inference into sequential steps), our work investigates specification decomposition. This involves transforming the instruction set itself into typed, independently-executable components.

This distinction matters: while reasoning decomposition preserves the full context at each step, specification decomposition deliberately fragments it. This fragmentation provides a benefit of variance compression by presenting each extraction slot with a simpler, less ambiguous task, yet it introduces a cost in the form of slot-filling hallucinations, as each individual slot loses the holistic perspective required to determine if its target truly exists within the input. Furthermore, the move toward structured output and typed generation through JSON mode [OpenAI, 2024] or Pydantic-based frameworks like the MELLEA library [IBM Research, 2025] enforces a necessary capability floor. By using schemas to constrain output shape, we can detect and repair worst-case formatting failures rather than silently propagate them downstream. Our proposed pipeline is built upon MELLEA.

Compiler-community grounding for robust code generation. A second, more recent thread of work treats LLM-driven code generation as a classical compiler-construction problem rather than a prompt-engineering one. Production code-generation agents converge on three load-bearing layers, external grounding (retrieved API documentation), deterministic verification (parser + lint + bounded repair), and narrow structured output, rather than on accumulating natural-language rules in a single prompt [Xia et al., 2024, Khattab et al., 2024]. Three pieces of evidence drive this convergence. First, retrieval over API documentation is consistently associated with the largest reported quality gains for libraries that postdate model training cutoffs, with small models on private or under-represented APIs benefiting most [Zhou et al., 2023b, Zhang et al., 2026]. Second, iterative repair with execution feedback yields large gains at small-model scale, especially where the oracle is a deterministic parser or lint output rather than another model judging itself [Chen et al., 2024]. Third, small models hit constraint-density and primacy-effect ceilings well before the dozens of rules a typical agent skill specification encodes [Jiang et al., 2024, Liu et al., 2024]. Engineering practice in established compiler tooling, with error-recovering parsers that report all violations in one pass, per-rule severity laddering so that one diagnostic does not gate the next, and version-predicated migration recipes that retire automatically when their target version arrives [Marsh and Astral, 2024, Moderne, Inc., 2024], supplies a vocabulary for expressing each of these correctness layers as a structured artifact consulted by both generation and verification. The compiler we describe in §3.1 adopts these three layers explicitly: a retrieved MELLEA API reference grounds code generation,

deterministic Python emitters write structured commitments rather than asking the model to author Python escape state, and a tiered static-validation pass feeds a bounded repair loop when its checks fail.

Natural language skill specifications have genuine strengths as a development format. They are accessible to domain experts who are not programmers, they support rapid iteration with immediate feedback, and they can be tested within minutes of being written. For prototyping and proof-of-concept work, this accessibility is a significant advantage. However, the same properties that make `.md` files effective for prototyping become liabilities in production deployment. There is no contract between the specification and the output, no mechanism to verify that requirements are met, and no way to ensure consistent behaviour across model updates or substitutions. *The gap between PoC and production is a chasm, not a step.* The MELLEA SKILLS COMPILER pipeline addresses this gap by providing an automated compilation step designed to preserve the original specification as the authoring format while producing a structured, validated, governed package for deployment; whether this design delivers in practice is the subject of an ongoing evaluation reported separately. The journey from idea to production follows a natural progression: domain experts describe the problem in plain language, prototypers build `.md` skills with fast iteration cycles, and developers run MELLEA-FY to transform the prototype into production-ready code with validation, error handling, and reliability guarantees.

2.4 Agent Skill Benchmarking

Benchmarking the accuracy and other performance metrics of agents to complete their intended actions as a function of their underlying skills is required to measure the progress of skill and model development. SkillsBench evaluates the effect of curated skills on agent task performance, finding that context overload from comprehensive documentation degrades performance by -2.9 percentage points, while focused 2–3 module skills improve performance by $+18.6$ percentage points [Li et al., 2026]. This finding converges with our perspective that structured, scoped specifications outperform monolithic ones under specific conditions.

Our planned evaluation differs from SkillsBench in three respects. First, we study *decomposition* of a single specification rather than *selection* among multiple skill modules. Second, we focus on small-model execution rather than frontier-model performance. Third, the design positions itself to give a mechanistic account of *why* decomposition helps (variance compression through typed schemas, externalised grounding) and *why* it hurts (slot-filling hallucination, context destruction) rather than to report aggregate performance differences alone.

Moreover, the Anthropic skill guide’s testing methodology recommends measuring performance comparison with vs. without skill, the same paired evaluation design we plan to use [Anthropic, 2025]. Our framework extends the Anthropic guide’s methodology by adding multi-dimensional scoring (NLI accuracy, structural completeness, specificity, completeness, utility) and by testing across models rather than considering a single target.

3. System Architecture

Agent skill specifications are natural language documents: markdown files containing instructions, constraints, examples, and domain knowledge, executed directly by language models at inference time. When a frontier model processes such a specification, it can resolve implicit contradictions, exercise holistic judgement across dozens of interacting rules, and produce coherent output. But deployment economics increasingly favour small models (less than $\sim 25\text{B}$ parameters), which lack

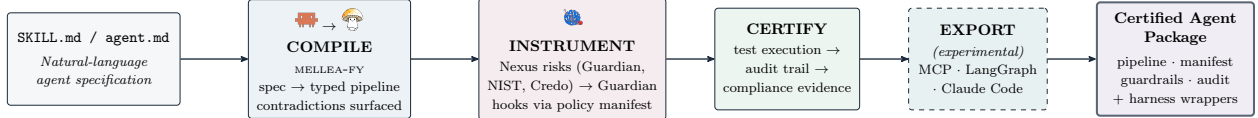


Figure 1: The MELLEA SKILLS COMPILER pipeline. A natural-language agent specification passes through four stages, the fourth being experimental: **Compile** (MELLEA-FY decomposes the spec into a typed MELLEA pipeline), **Instrument** (AI Atlas Nexus surfaces the risks applicable to this skill across Guardian, NIST, and Credo taxonomies, and the corresponding Guardian hooks attach to the pipeline via the policy manifest), **Certify** (test execution produces runtime evidence and a compliance report), and **Export** (re-target the certified package to MCP, LangGraph, or Claude Code without re-implementation; experimental). The output is a certified agent package with embedded guardrails and an auditable provenance trail, optionally accompanied by harness-specific deployment wrappers.

these capabilities. The result is a widespread mismatch: specifications authored for frontier reasoning, executed by models with substantially reduced capability.

MELLEA SKILLS COMPILER addresses this mismatch through a four-stage pipeline (Figure 1) that transforms a raw skill specification into a certified, governed agent package, the fourth stage being an experimental *export* step that re-targets the certified package to alternative deployment harnesses. Each stage addresses a distinct concern:

1. **Compile**: decompose the natural language spec into a typed, multi-step MELLEA program, trading holistic prompt execution for focused, schema-constrained generation steps.
2. **Instrument**: identify the risks applicable to this skill’s use case across the governance taxonomies covered by AI Atlas Nexus (currently including Guardian, NIST, and Credo), and attach the corresponding Guardian guardrails to the pipeline via MELLEA’s hook system, so every LLM generation is automatically checked at runtime.
3. **Certify**: execute the instrumented pipeline on test fixtures, collect runtime evidence, and produce compliance documentation.
4. **Export** (experimental): produce target-specific adapters so the certified MELLEA package can be invoked from external hosts (MCP servers, LangGraph nodes, Claude Code skills) without re-implementation.

The pipeline’s central architectural property is *what gets compiled is what gets certified is what gets deployed*. The same code runs in evaluation, certification, and production; Guardian attaches as an observer, not a wrapper. The Export stage extends this property: each export is a thin adapter that delegates to the certified MELLEA package’s entry point, so the behaviour that was certified is the behaviour that runs, whichever host invokes it.

3.1 Stage 1: Compile — Specification to Typed Pipeline

The first stage transforms a natural language specification into an executable MELLEA pipeline package. The transformation is performed by MELLEA-FY, a structured seven-step workflow that bounds and scopes LLM invocations (Figure 2), implemented as a Claude Code command that reads the specification and emits a complete Python package. Each LLM call is restricted to a single narrow task (element tagging, mapping judgements, body generation, narrative prose); deterministic Python writes the surrounding scaffolding (file emission, dependency audit, lint pass, repair routing). The workflow’s *control flow* is deterministic; the LLM *outputs* are not, and the compiler does not claim run-to-run byte-level reproducibility (see §5).

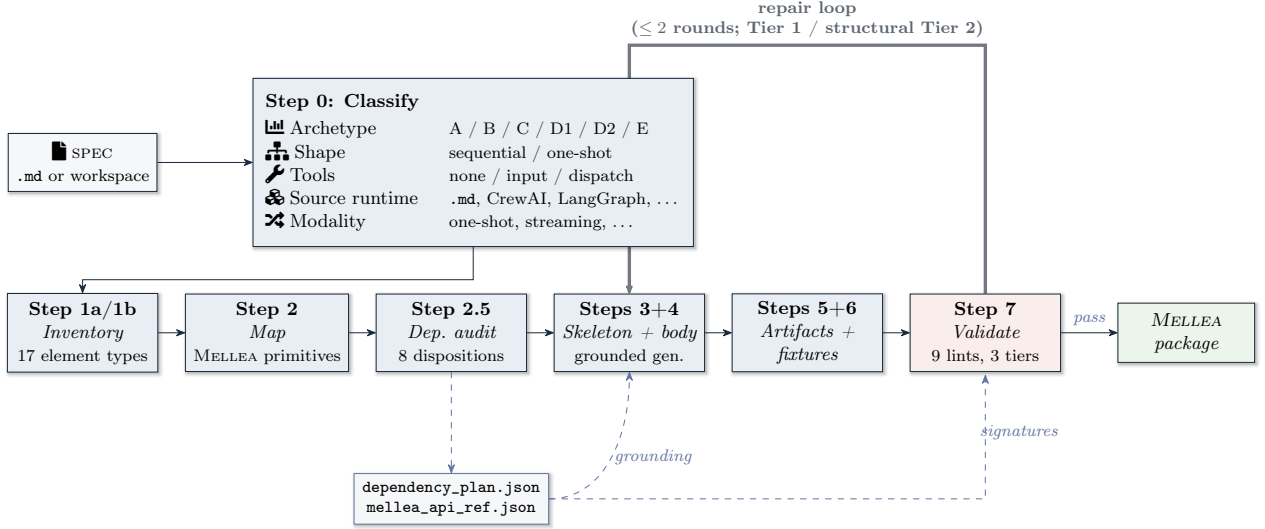


Figure 2: The MELLEA-FY compilation pipeline. Step 0 classifies the specification along five axes; the classification informs every subsequent step. LLM invocations are confined to a small set of narrow tasks; surrounding scaffolding is deterministic Python. Step 2.5 produces two artifacts that act as a single source of truth with two consumers: a dependency plan and an introspected MELLEA API reference, both consumed by Steps 3+4 (as grounding context for code generation) and by Step 7 (for the cross-reference and signature lints). Tier-1 and structural Tier-2 lint failures at Step 7 route the failing files back to Steps 3+4 in a bounded repair loop.

Design rationale

The design aims to address a practical problem: a typical agent skill encodes thirty or forty rules, more than a small task model can reliably follow at execution time. Recent work shows small models hit constraint-density and primacy-effect ceilings well before this point [Jiang et al., 2024, Liu et al., 2024], and that demonstrations carry behavioural specification more reliably than prose at this scale [Min et al., 2022]. Decomposing the spec into a typed multi-step pipeline, where each step is a focused generation request, addresses the runtime side of this problem.

The compile step itself currently runs on a frontier-class model, but borrows architectural patterns from production code-generation research: ground generation in retrieved API documentation, verify the result with deterministic checks, bound LLM invocations narrowly [Xia et al., 2024, Zhou et al., 2023b]. MELLEA is a relatively new generative-computing coding paradigm, so its presence in model training data is limited; this makes the grounding move particularly applicable here. These patterns improve the compiler’s robustness regardless of compile-time model size, and they are a precondition for future work that might run some compile steps on smaller models [Khatab et al., 2024].

The seven steps

Step 0 — Five-axis classification. Before any other step runs, the spec is classified along five axes that drive subsequent decisions: the *reasoning archetype* the skill performs, the *pipeline shape* it implies, how *tools* are involved, the *source runtime* the spec is written for, and the *interaction modality* the runtime expects. The classification drives every later step: which files the inventory pass reads, which mapping table applies, what entry point the generated package exposes. Source-runtime detection uses weighted signal scoring with explicit tiebreakers, mirroring the production pattern of static capability tables consulted at both generation and verification time [Gauthier, 2024]. The full

set of values for each axis, with detection signals and downstream effects, is given in Appendix A.

Step 1 — Inventory. The spec is read in full; every significant element is tagged with the kind of work it represents (extraction, classification, validation, generation, deterministic transformation, and so on) and assigned to a dependency category that captures how it relates to the world outside the pipeline (persona text, operating rules, user facts, runtime state, tools, credentials, and so on). The full element-type catalogue (seventeen values) and the dependency-category catalogue (nine values) are given in Appendix A. Source fidelity is non-negotiable: at least 95% of significant lines become inventory entries. The file scan itself is deterministic; tagging uses a single, scoped LLM invocation per file.

Step 2 — Element-to-primitive mapping. Each element is mapped to a MELLEA construct, choosing between validators, generative slots, structured-output instructions, and deterministic helpers according to the kind of work the element does. Two distinctions matter most. First, the compiler separates *output validation* (checks on the agent’s own generated text) from *domain validation* (checks on external artifacts a tool produces). These differ in how they are implemented: an output validator runs as a Python check on the generated string; a domain validator uses a two-step pattern that first extracts structured data and then assesses it. Second, schema complexity governs the choice of generation strategy: simple cases use the lightest typed-output construct directly; complex schemas use a repair-template fallback for malformed JSON, a necessity for small task models that frequently produce truncated output. The full mapping table is in Appendix A.

Step 2.5 — Dependency audit and MELLEA API reference. Every external dependency is assigned an explicit *disposition*: a commitment about how the compiler should handle it. A dependency might be embedded inline in the generated code as a constant, generated as a real implementation, left as a placeholder for the human integrator to complete, mocked with fixture data for testing, delegated to the host runtime, supplied at invocation time as a configuration value, loaded from disk at runtime, or excluded entirely. Eight disposition values cover these cases; the full set with semantics and category defaults is given in Appendix A, Table 5.

How a disposition is chosen depends on the elicitation mode. By default the compiler applies category-based defaults silently; an interactive mode opens a terminal UI so the user reviews each dependency and accepts or overrides the default; a strict mode halts before any code is emitted if any dependency would resolve to a placeholder. Dispositions are the compiler-tooling analog of versioned migration recipes [Moderne, Inc., 2024]: explicit, gate-able, retire-able commitments.

This step also introspects the installed MELLEA package and writes a slice of its public API surface to a small JSON file. The slice serves two consumers: the body-generation step (Step 4) reads it as grounding context, and the static-validation pass (Step 7) reads it as the live signature source for the lint that checks MELLEA stdlib calls. One source of truth, two consumers. When MELLEA is not importable in the compile environment, the file records that state and the downstream steps fall back to a static signature table and the catalogued Known-Behaviour patterns. Step 2.5 itself contains no LLM invocations.

Steps 3 + 4 — Skeleton emission and grounded body generation. Step 3 emits skeleton files (imports, function signatures, docstring placeholders) deterministically from the element mapping and dependency plan. Which files are emitted depends on the classification: every package gets a core set covering the pipeline orchestration, the schemas, the configuration constants, the entry point, and the project metadata, with additional files added when the inventory or disposition plan requires

them (extraction slots, tool definitions, requirement validators, on-disk loaders). The full file-set decision is in Appendix A. Constants and configuration values are written by deterministic Python rather than asked of the LLM, so the model emits structured values and the compiler synthesises the source-level literal [Microsoft, 2023].

Step 4 invokes the LLM once per code body, with grounding context drawn from the API-reference slice produced at Step 2.5 and from the catalogued small-model failure patterns relevant to the detected runtime. The doc-augmented framing follows the pattern associated with the largest documented quality gains on libraries with limited training-data presence [Zhou et al., 2023b, Wang et al., 2025b], an approach from which smaller models in particular benefit [Zhang et al., 2026] and which therefore remains viable if compile-time work later moves to smaller models.

Worked example: a compiled weather skill

To make the output of Steps 3+4 concrete, Listing 1 shows abridged fragments of three generated files for a small `weather` skill whose source spec is a ~60-line `SKILL.md` describing how to fetch weather conditions from a public HTTP service. The fragments illustrate four architectural properties at once: (i) typed schemas in `schemas.py` carry the contract between phases (`WeatherIntent`, `WeatherReport`); (ii) a `@generative` slot in `slots.py` encodes the extraction step as a Python function signature; (iii) the pipeline orchestrates two phases in *separate* MELLEA sessions, enforcing the one-Pydantic-schema-per-session discipline that the `session-boundary` lint will check at Step 7; and (iv) deterministic helpers (`_parse_instruct_result`, `_build_weather_url`, the URL-template constant) sit between LLM calls in `pipeline.py`, with the constant emitted by the deterministic literal writer described in Step 3.

Listing 1: Abridged fragments from the compiled `weather` skill: a typed schema (top), an `@generative` extraction slot (middle), and a two-phase pipeline (bottom). Comments and line continuations are preserved as generated.

```
# schemas.py -- typed contract between phases
class WeatherIntent(BaseModel):
    location: str = Field(..., description="City, region, or airport code")
    query_type: str = Field("current",
        description="One of: current, forecast, tomorrow, rain, wind")

class WeatherReport(BaseModel):
    location: str
    conditions: str
    summary: str = Field(..., description="One-sentence plain-language summary")

# slots.py -- Step 4 emits the body for an extraction slot
@generative
def extract_weather_intent(user_query: str) -> WeatherIntent:
    """Extract weather query intent from the user's question.
    Set query_type to one of: current, forecast, tomorrow, rain, wind."""
    ...

# pipeline.py -- two-phase orchestration in separate sessions (KB5)
def run_pipeline(user_query: str) -> str:
    # Phase 1: extract intent
    with start_session(BACKEND, MODEL_ID) as m:
        intent = extract_weather_intent(m, user_query=user_query)

    # Deterministic step between phases
    url = _build_weather_url(intent.location, intent.query_type)
```

```

raw_data = fetch_weather(url)

# Phase 2: format -- new session because the schema changes
with start_session(BACKEND, MODEL_ID) as m2:
    report_thunk = m2.instruct(
        "Summarize this weather data clearly for the user.",
        model_options={ModelOption.SYSTEM_PROMPT: PREFIX_TEXT},
        grounding_context={"raw_weather_data": raw_data,
                           "location": intent.location},
        format=WeatherReport,
        strategy=RepairTemplateStrategy(loop_budget=LOOP_BUDGET),
    )
    report = _parse_instruct_result(report_thunk, WeatherReport)

return f"{report.location}: {report.conditions} -- {report.summary}"

```

The full compiled package contains nine files (`pipeline.py`, `slots.py`, `schemas.py`, `config.py`, `tools.py`, `main.py`, an `__init__` and `__main__` entry, and a `pyproject.toml` at the skill root) plus an `intermediate/` directory with the per-step artifacts. None of these files exceed 75 lines, and every line is traceable back to a source-spec element through `intermediate/element_mapping.json`.

Step 5 — Supporting artifacts. A `mapping_report.md` records the classification with evidence and an element-level mapping table linking each specification element to its generated file and function. A `melleafy.json` manifest captures the detected source runtime and modality, the resolved dependency dispositions, the declared environment variables, and the entry-point signature. When the package contains stubs or runtime-delegated dependencies, a `SETUP.md` documents what the human integrator must complete before deployment. For non-Markdown source runtimes, a `SKILL.md` CLI-compatibility shim is generated.

Step 6 — Fixtures. Five to eight executable fixtures are generated, covering at least three different dependency categories. The fixtures double as the semantic oracle for downstream verification, instantiating the production-tested pattern of using executable tests as the primary correctness signal [Chen et al., 2022, Ridnik et al., 2024].

Step 7 — Static validation with bounded repair. The workflow’s final gate is a static-validation pass partitioned into three tiers (Figure 2). The first tier catches syntactic problems: every generated `.py` file must parse, and the package’s entry module must import cleanly in a subprocess. The second tier runs a set of structural checks in parallel: that every mapped target symbol resolves, that validator signatures and bodies are sound, that each MELLEA session uses at most one Pydantic schema (a discipline that small models occasionally violate), that imports are real, that calls to known MELLEA stdlib functions match their introspected signatures, and that the catalogued small-model failure patterns are absent. The third tier runs only when the structural tier is clean and covers cross-artifact concerns (entropy scans for leaked credentials, manifest consistency). The full lint catalogue, with what each lint defends and how its failures route, is given in Appendix A, Table 6.

This three-tier shape is the standard error-recovering, per-rule-isolated production lint pattern: a parser that recovers and reports all violations in one pass, with severity laddering so one diagnostic does not gate the next [Marsh and Astral, 2024, Cui et al., 2024]. Tier-1 and structural Tier-2 failures trigger a *bounded repair loop*: the failing files are re-passed to the body-generation step (Step 4) with the exact lint message and a three-line context window, capped at two repair rounds before the run

halts. This is the setting in which iterative repair with parser/lint feedback shows the largest reported improvements at small-model scale [Chen et al., 2024], and where its known limitations on logical errors [Olausson et al., 2024] are sidestepped because the oracle is deterministic, not another model judging itself. Two lint classes are deliberately excluded from the repair loop: **session-boundary** failures (architectural mistakes that require regenerating the full file) and **category-specific** failures (leaked secrets and other security concerns where silent re-generation is inappropriate).

The compile stage as a whole

The pipeline organises correctness around three load-bearing layers, each consulted by both generation and verification: a static capability table (the five-axis classification at Step 0); a retrieved API reference (`mellea_api_ref.json` at Step 2.5); and a verification-and-repair loop with deterministic checks (Step 7). The same JSON file that grounds Step 4 also drives Step 7’s **stdlib-arity** lint; the same dependency plan that drives Step 3’s file set also drives Step 7’s **cross-reference** lint. Each commitment in the pipeline is a structured artifact with an owner, a default, an audit record, and (for compatibility shims keyed to particular MELLEA versions) a version predicate at which it expires [Moderne, Inc., 2024]. Full details of the classification axes, element types, dependency categories, the eight dispositions, and the per-lint sub-checks are provided in Appendix A.

3.2 Stage 2: Instrument — Use Case to Policy-Driven Guardrails

The second stage takes the compiled pipeline and instruments it with runtime guardrails. The key insight is that the skill’s use-case description, read directly from the source specification’s metadata (the **description**: frontmatter field for Markdown skills, or its runtime-specific equivalent) and surfaced into `melleafy.json` during Step 5, can be submitted to a governance knowledge graph to automatically identify applicable risks, without manual risk assessment.

AI Atlas Nexus [Vejsbjerg et al., 2026] receives the use-case description and returns risks from whichever taxonomies it covers; current taxonomies include Granite Guardian (runtime safety checks), NIST AI RMF (governance controls), and Credo UCF (trustworthiness criteria). Each Guardian risk is classified as either *native* (using Guardian’s calibrated detection for dimensions like harm, jailbreak, or social bias) or *custom* (free-form criteria for risks outside Guardian’s native taxonomy). Governance risks are expanded into actionable requirements (for example, NIST control **GV-1.2-001**) with cross-taxonomy links carried by Nexus.

The resulting **PolicyManifest** drives the creation of MELLEA hook plugins. Two modes are supported: *audit* (observe and log every Guardian verdict without blocking) and *enforce* (halt the pipeline when a risk is detected). Because guardrails attach via MELLEA’s hook system, they intercept every `m.instruct()` and `@generative` call automatically; no modification to the pipeline code is required. An audit trail plugin records every pipeline event (generations, tool calls, Guardian verdicts, validation results) as a JSONL log linked back to the policy manifest.

Before policy generation, the skill’s declared tools are classified into sensitivity tiers (non-sensitive, sensitive, restricted) via a static lookup table. The resulting sensitivity feeds into the use-case description, influencing which risks Nexus identifies. Full details of the manifest structure, hook architecture, and sensitivity classification are provided in Appendix B.

3.3 Stage 3: Certify — Runtime Evidence to Compliance Documentation

With a compiled pipeline (Stage 1) and a policy manifest with Guardian hooks (Stage 2) in place, the certification stage runs the pipeline on test fixtures and collects runtime evidence. The MELLEA

SKILLS COMPILER `certify` command executes four steps beyond what Stages 1 and 2 have already produced:

1. **Execute with hooks active.** The compiled pipeline runs on a selected test fixture with Guardian and audit trail plugins registered. Because Guardian attaches via MELLEA’s hook system, every `m.instruct()` and `@generative` call inside the pipeline is automatically checked; no modification to the pipeline code is required. In *audit* mode, Guardian logs verdicts without blocking; in *enforce* mode, a flagged risk raises a `PluginViolationError` that halts the pipeline.
2. **Verdict aggregation.** Guardian verdicts across the run are summarised: total checks, passed (no risk detected), and flagged (risk detected). Each flagged verdict records the risk name and a preview of the triggering output. The JSONL audit trail captures every pipeline event (generation calls, tool invocations, Guardian verdicts, and validation results) with timestamps and session identifiers.
3. **Compliance classification.** Each governance action from the policy manifest (§3.2) is classified as `AUTOMATED` (covered by Guardian checks or structural validators), `PARTIAL` (partially addressed), or `MANUAL` (requires human review). This classification uses the runtime evidence to determine which governance controls (across all the taxonomies AI Atlas Nexus covers) are demonstrably satisfied by the pipeline’s behaviour.
4. **Certification report.** A human-readable `CERTIFICATION.md` consolidates the policy manifest, Guardian verdicts, audit trail summary, and compliance classification into a single reviewable document.

The certification stage adds three artifacts to the skill’s `audit/` directory (alongside the `manifest.json` and `POLICY.md` produced in Stage 2):

Artifact	Purpose
<code>pipeline_report.json</code>	Structured pipeline output for the test fixture
<code>audit_trail.jsonl</code>	Runtime event log (every LLM call + Guardian verdict)
<code>CERTIFICATION.md</code>	Consolidated certification report

The key architectural property is that the same compiled code runs in both evaluation and certification. The pipeline is not re-implemented or wrapped; Guardian attaches via MELLEA’s hook system and observes the pipeline’s actual runtime behaviour. This ensures that what is certified is what is deployed; the optional Export stage (§3.4) carries the same property across alternative deployment harnesses, because the exported wrappers delegate to the certified package rather than re-implement it.

3.4 Stage 4: Export — Re-targeting to Deployment Harnesses (experimental)

The Stages 1–3 output is a self-contained MELLEA Python package. To make that package callable from hosts other than the MELLEA CLI itself, a Model Context Protocol [Anthropic, 2024] server, a LANGGRAPH node, or a Claude Code skill, `melleafy export` produces a target-specific adapter. The Mellea package itself is unchanged; each export is a thin entry point that delegates to it.

This delegation discipline is deliberate. The exports do *not* re-translate the pipeline into the target framework’s native idiom; the certified Python is what runs. This preserves the *what is certified is what is deployed* property across hosts: a Guardian verdict observed during certification is observed identically when the same pipeline is invoked through MCP, because the certified code itself is what runs in both cases. The pattern (typed input/output contracts, deterministic adapter

generation) follows production domain-specific languages for LLM function calling [Microsoft, 2023, BoundaryML, 2024].

Three exports are available today:

- **MCP server export.** Exposes the pipeline as one or more typed tools on an MCP server. The skill's `run_pipeline` signature determines the tool input schema (one tool per skill, or one per declared entry when the skill exposes multiple); the modality classification (Step 0 axis 5) determines whether the tool returns synchronously or streams incremental output.
- **LangGraph node export.** A callable node suitable for inclusion in a `LANGGRAPH StateGraph`. The node accepts and returns a `LANGGRAPH` state object whose schema is derived from the pipeline's input and output Pydantic models.
- **Claude Code skill export.** A Claude Code skill bundle with a generated `SKILL.md` (already produced by Step 5 for non-Markdown source runtimes; reused here) and a small invocation script that the Claude Code skill loader recognises.

All three exports are flagged *experimental*: the adapter contracts are stable enough for internal use but await broader testing across the skill matrix and across versions of the target hosts. §4 discusses the path to maturity.

4. Ongoing and Future Work

The architecture described in §3 anticipates several extensions that are under active development. Each connects to a mechanism the present report has already named.

Empirical evaluation. The architecture introduces several mechanisms whose contribution requires empirical measurement. An evaluation is underway and will be reported separately.

Interactive dependency resolution during compile. Step 2.5 currently supports three elicitation modes for dependency dispositions: `auto` (apply category defaults silently), `ask` (a terminal UI for per-dependency review), and `strict` (halt before emitting code if any disposition would produce a stub). In production deployments, where a human integrator must make these decisions iteratively (reviewing a stub-pile, deciding which to implement now versus defer, registering credentials at the same time), the present terminal UI is too coarse. A richer interactive flow would (i) cluster dispositions by category and surface them in priority order, (ii) accept partial commitments that the compiler can fall back from later, (iii) integrate credential and tool registration alongside disposition choice, and (iv) record the human-resolved decisions in `melleafy.json` so subsequent re-compiles can restore them without re-prompting.

Maturing the Export stage. The MCP, LangGraph, and Claude Code exports described in §3.4 are functional today but experimental. Maturing them entails (i) a coverage matrix of which Step 0 modalities each harness supports natively (LangGraph's streaming model differs from MCP's, for example), (ii) round-trip behavioural-equivalence tests so a pipeline's Guardian verdicts under the native MELLEA runtime are reproduced under each export target on the same fixtures, and (iii) versioning of the wrapper contracts against each upstream harness so a Claude Code skill exported today continues to load when Claude Code's loader changes. Additional export targets (OpenAI Agents SDK, AutoGen, smolagents) are candidates once the existing three are stable.

Maturing non-Markdown source-runtime support. The Step 0 source-runtime axis (§3.1) lists nine recognised input formats. Markdown skill specifications are mature; the others are recognised by

the classifier and routed to dialect-specific inventory passes, but the per-dialect mapping tables and Known-Behaviour catalogues are at varying levels of completeness. Maturing each non-Markdown runtime requires a per-runtime fixture suite and a regression matrix run on each MELLEA release; this work is planned alongside the re-evaluation, since the eval plan already requires representative skills in CrewAI, LangGraph, and Letta.

Increased coverage for additional interaction modalities. The Step 0 modality axis declares eight modalities, and the generated entry-point shape follows from the modality classification. The `synchronous_oneshot` modality is mature today. Streaming, conversational session, scheduled, event-triggered, heartbeat, and real-time-media modalities are recognised by the classifier and produce the corresponding entry-point shapes, but their behaviour under Guardian-instrumented runs and under Step 7’s lint pass is less thoroughly exercised. Recent MELLEA additions, `stream_validate()` on the `Requirement` class and streaming support in MELLEA’s OpenAI-compatible API server, make principled modality-specific verification newly tractable, in particular for the `streaming` and `conversational_session` modalities. Closing the rest of the matrix is incremental work that the re-evaluation will help prioritise.

5. Limitations

This report describes an architecture and a design rationale; the limitations follow from what the report does not yet contain.

Empirical claims await evaluation. This report describes architecture and design rationale. Output-quality numbers, decomposition-benefit estimates, and model-tier substitution claims are not established here; an evaluation is underway and will be reported separately (§4).

Run-to-run output is not bit-for-bit reproducible. The compiler invokes an LLM for element tagging (Step 1b), narrow mapping judgements (Step 2), body generation (Step 4), and narrative artifact prose (Step 5). These calls are not deterministic; the same source spec can produce different compiled packages on different runs. The mitigations are scope (each invocation is bounded to a narrow task), validation (Step 7 catches errors regardless of which valid solution the LLM produced), and provenance (`intermediate/element_mapping.json` records the mapping decisions for every element, so drift between runs is auditable). The compiler does not claim byte-level reproducibility. A related effect is that `melleafy` runs as a Claude Code command; the LLM’s context can include other files in the surrounding repository, so the compiler’s output may be influenced by existing patterns in the user’s codebase. Whether this is an asset (consistency with the user’s own conventions) or a liability (cross-user variability) depends on what the repository contains.

Compilation faithfulness is a new metric without an external benchmark. The fraction of source-spec elements whose mapping survives the seven-step workflow is a useful internal measure but has no established external reference value. Until the re-evaluation supplies its first measurements, the metric should be read as a structural diagnostic (whether the compiler is preserving spec content) rather than as a calibrated quality score.

The disposition taxonomy and the lint count are design choices. The eight dependency dispositions and the partition of nine lints into three tiers are this work’s contributions; they are positioned in the report against compiler-community precedent (version-predicated migration recipes,

error-recovering parsers, per-rule severity laddering), but they are not derived from external literature. A reader who finds a different taxonomy more natural for their own setting is unlikely to find a published refutation of either choice; the partitions reflect engineering judgement informed by the small-model code-generation evidence cited.

Per-model failure-rate claims would require our own measurements. The literature cited in §2–§3 establishes the *setting* in which our design choices apply (small-model scale, under-represented libraries, instruction-density ceilings), but it does not give Granite-3.3-8B-specific or Granite-4.0-Hybrid-Tiny-specific failure rates. Where the report makes claims about small-model behaviour, those claims should be read as scoped to that setting, not to the particular task models we plan to evaluate.

Export-stage coverage is partial. The MCP, LangGraph, and Claude Code targets described in §3.4 are experimental. Round-trip behavioural-equivalence tests across all four pipeline stages and across the eight modalities have not yet been exhaustively run; the maturation path is described in §4.

6. Conclusion

The current wave of skills expressed in natural-language Markdown files, authored and shared by a broad community, has lowered the barrier to participation for both technical and non-technical users and contributed to their widespread adoption within organisations. Their greatest strength lies in prototyping and rapid iteration.

When these skills are promoted to production, particularly in business-critical contexts, their limitations become evident. Deploying logic as `.md` files often bypasses formal verification, runtime monitoring, and compliance guarantees. Natural-language instructions can be fragile, breaking silently as context or underlying models change, with failures detected only through downstream impact. Security concerns compound these risks: there are limited mechanisms for runtime safety assessment during execution.

MELLEA SKILLS COMPILER addresses these challenges by compiling skills into typed, validated programs with guardrails, monitoring, and auditable execution embedded in the runtime. This shifts skills from informal, best-effort artifacts into governed, observable, and production-ready components. A regulated-industry deployment, for example, requires both runtime guardrails and an auditable execution trace; the certified package this pipeline produces supplies both as primary outputs rather than as add-ons.

Significant open questions remain, and the architecture awaits rigorous empirical evaluation, which is underway and will be reported separately. We believe this work represents a meaningful step toward production-ready agentic skills, one in which the gap between a prototype `.md` file and a deployable, certified package can be addressed systematically.

References

Alex and Oren Yomtov. ClawHavoc: 341 malicious clawed skills found by the bot they were targeting. <https://www.koi.ai/blog/clawhavoc-341-malicious-clawedbot-skills-found-by-the-bot-they-were-targeting>, 2026.

- Anthropic. Model context protocol. <https://modelcontextprotocol.io/>, 2024. Open specification for connecting AI assistants to external tools and data sources.
- Anthropic. The complete guide to building skills for Claude. Technical report, Anthropic, 2025. URL <https://resources.anthropic.com/hubfs/The-Complete-Guide-to-Building-Skill-for-Claude.pdf>. PROVENANCE: user-provided PDF — verified via direct download 2026-03-08.
- BoundaryML. BAML: A domain-specific language for LLM function calling. <https://github.com/BoundaryML/baml>, 2024. needs-confirmation on canonical citation form.
- Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. CodeT: Code generation with generated tests. *arXiv preprint arXiv:2207.10397*, 2022.
- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2304.05128.
- Ana Maria Constantin. Anthropic cuts Claude subscribers off from OpenClaw in cost crackdown. <https://thenextweb.com/news/anthropic-openclaw-claude-subscription-ban-cost>, April 2026.
- Han Cui, Menglei Xie, Ting Su, Chengyu Zhang, and Shin Hwei Tan. An empirical study of false negatives and positives of static code analyzers from the perspective of historical issues. *arXiv preprint arXiv:2408.13855*, August 2024.
- Paul Gauthier. Aider: AI pair programming in your terminal. <https://aider.chat>, 2024. includes “Unified diffs make GPT-4 Turbo 3X less lazy,” May 2024.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM workshop on artificial intelligence and security*, pages 79–90, 2023.
- Finn Hillebrandt. OpenClaw statistics 2026: Key numbers, data & facts. <https://www.gradually.ai/en/openclaw-statistics/>, April 2026.
- Florian Holzbauer, David Schmidt, Gabriel Gegenhuber, Sebastian Schrittwieser, and Johanna Ullrich. Malicious or not: Adding repository context to agent skill classification. arXiv:2603.16572, 2026.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *The twelfth international conference on learning representations*, 2023.
- IBM Research. Mellea: A generative programming library for LLM workflows. <https://github.com/generative-computing/mellea>, 2025.
- Yuxin Jiang, Yufei Wang, Xingshan Zeng, Wanjun Zhong, Liangyou Li, Fei Mi, Lifeng Shang, Xin Jiang, Qun Liu, and Wei Wang. FollowBench: A multi-level fine-grained constraints following benchmark for large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2310.20410.

- Samuel GB Johnson, Amir-Hossein Karimi, Yoshua Bengio, Nick Chater, Tobias Gerstenberg, Kate Larson, Sydney Levine, Melanie Mitchell, Iyad Rahwan, Bernhard Schölkopf, et al. Imagining and building wise machines: The centrality of ai metacognition. *Trends in Cognitive Sciences*, 2025.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.03714.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, et al. SkillsBench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026. PROVENANCE: verified via arxiv.org 2026-03-08.
- Paulina Likos. Nvidia CEO Jensen Huang says OpenClaw is definitely the next Chat-GPT. <https://www.cnbc.com/2026/03/17/nvidia-ceo-jensen-huang-says-openclaw-is-definitely-the-next-chatgpt.html>, March 2026.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics (TACL)*, 2024. arXiv:2307.03172.
- Charlie Marsh and Astral. Ruff: An extremely fast Python linter, written in Rust. <https://github.com/astral-sh/ruff>, 2024. v0.4 introduced an error-recovering hand-written parser.
- Microsoft. TypeChat: Build natural language interfaces using types. <https://github.com/microsoft/TypeChat>, 2023.
- Sewon Min, Xinxu Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. Rethinking the role of demonstrations: What makes in-context learning work? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022. arXiv:2202.12837.
- Moderne, Inc. OpenRewrite: Automated source code refactoring. <https://docs.openrewrite.org>, 2024.
- Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2306.09896.
- OpenAI. Json mode. <https://platform.openai.com/docs/guides/structured-outputs>, 2024. PROVENANCE: claude-suggested — verify URL is current.
- Inkit Padhi, Manish Nagireddy, Giandomenico Cornacchia, Subhajit Chaudhury, Tejaswini Pedapati, Pierre Dognin, Keerthiram Murugesan, Erik Miehl, Martín Santillán Cooper, Kieran Fraser, Giulio Zizzo, Muhammad Zaid Hameed, Mark Purcell, Michael Desmond, Qian Pan, Inge Vejsbjerg, Elizabeth M. Daly, Michael Hind, Werner Geyer, Ambrish Rawat, Kush R. Varshney, and Prasanna Sattigeri. Granite guardian: Comprehensive llm safeguarding. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track)*, pages 607–615, 2025.

- Tal Ridnik, Dedy Kredo, and Itamar Friedman. Code generation with AlphaCodium: From prompt engineering to flow engineering. *arXiv preprint arXiv:2401.08500*, 2024.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
- Peter Slattery, Alexander K. Saeri, Emily A. C. Grundy, Jess Graham, Michael Noetel, Risto Uuk, James Dao, Soroush Pour, Stephen Casper, and Neil Thompson. The AI risk repository: A meta-review, database, and taxonomy of risks from artificial intelligence. *Patterns*, page 101517, 2026.
- Matthew S. Smith. The first social network for AI agents heralds their messy future. *IEEE Spectrum*, February 2026. URL <https://spectrum.ieee.org/moltbook-agentic-ai-agents-openclaw>.
- Seshu Tirupathi, Dhaval Salwala, Elizabeth M. Daly, and Inge Vejsbjerg. GAF-Guard: An agentic framework for risk management and governance in large language models. *arXiv:2507.02986*, 2025.
- Inge Vejsbjerg, Rahul Nair, Elizabeth Daly, Dhaval Salwala, and Seshu Tirupathi. Risk atlas nexus: A system for managing ai risks. In *AAAI Conference on Artificial Intelligence*, 2026.
- Haoyu Wang, Christopher M Poskitt, and Jun Sun. Agentspec: Customizable runtime enforcement for safe and reliable llm agents. *arXiv preprint arXiv:2503.18666*, 2025a.
- Zora Zhiruo Wang, Akari Asai, Xinyan Velocity Yu, Frank F. Xu, Yiqing Xie, Graham Neubig, and Daniel Fried. CodeRAG-Bench: Can retrieval augment code generation? In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025b. *arXiv:2406.14497*.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022. PROVENANCE: well-known reference — verify page numbers.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First conference on language modeling*, 2024.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.
- Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning. *arXiv preprint arXiv:2406.09187*, 2024.
- Yitong Zhang, Chengze Li, Ruize Chen, Guowei Yang, Xiaoran Jia, Yijie Ren, and Jia Li. To see is not to master: Teaching LLMs to use private libraries for code generation. *arXiv preprint arXiv:2603.15159*, March 2026.
- Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*, 2023a. PROVENANCE: well-known reference — verify year (ICLR 2023).

Shuyan Zhou, Uri Alon, Frank F. Xu, Zhengbao Wang, Zhiruo Jiang, and Graham Neubig. DocPrompting: Generating code by retrieving the docs. In *International Conference on Learning Representations (ICLR)*, 2023b. arXiv:2207.05987.

A. Compilation Details

This appendix provides the full technical details of the MELLEA-FY compilation process summarised in Section 3.1.

A.1 Reasoning Archetype Classification

Table 1 defines the six reasoning archetypes and their decomposition strategies.

Table 1: Reasoning archetype classification (Axis 1).

Type	Name	Description	Decomposition Strategy
A	Analysis	Sequential phases → structured findings	Full slot decomposition, two-step enrichment
B	Generation	Intent + constraints → artifact	Intent capture + draft generation
C	Diagnosis	Hypothesis-driven investigation	Reasoning decomposition only
D1	Integration	Thin wrapper around single service	Intent classification layer only
D2	Orchestration	Multi-step tool/agent coordination	Decision logic only
E	Knowledge	Passive rules/procedures	Depends on internal structure

For Type E skills, the pipeline shape (sequential vs. one-shot) is determined by assessing the specification’s internal structure. Named stages with distinct purposes and inter-stage data dependencies indicate sequential shape (e.g., doc-coauthoring has three stages where context gathering feeds refinement feeds reader testing). A flat set of rules or guidelines with no sequential dependencies indicates one-shot shape (e.g., frontend-design applies all aesthetic principles simultaneously).

A.2 Element Tagging Inventory

The compiler reads the specification and tags every discrete instructional element with one of 17 element types:

Tag	Description
EXTRACT	Pull structured data from unstructured input
CLASSIFY	Categorisation or labelling task
GENERATE	Produce new text/content meeting specifications
VALIDATE_OUTPUT	Check on the agent’s own generated text
VALIDATE_DOMAIN	Check on external artifacts (code, data)
TRANSFORM	Convert data from one form to another
QUERY	Answer a question about provided data
DECIDE	Conditional branching based on evaluation
ORCHESTRATE	Multi-step workflow coordination
CONVERSE	Interactive/conversational element
REMEDiate	Iterative fix-and-verify loop
SCHEMA	Structured output format definition
CONFIG	Threshold, parameter, or constant
TOOL_TEMPLATE	Parameterised tool invocation template
DETERMINISTIC	Task that should be pure Python, not LLM
TOOL_INPUT	Data that must be gathered by tools before pipeline runs
NO_DECOMPOSE	Persona/style guidance that stays as prompt text

Compound elements are split (e.g., “identify and rule out failure modes” becomes **EXTRACT** + **VALIDATE_DOMAIN**), and a Python-vs-LLM decision is applied: tasks requiring only parameter mapping, string formatting, or numerical computation are tagged **DETERMINISTIC** and become plain Python functions rather than LLM calls. This inventory typically identifies 40–60 elements per specification.

In parallel with element tagging, every element is also assigned to one of nine *dependency categories* that capture how it relates to the world outside the pipeline. The category determines the default disposition the compiler applies at Step 2.5 (Table 5).

Table 2: Dependency categories assigned during inventory.

Code	Category	Examples
C1	Identity	Persona text, behavioural framing, role description
C2	Operating rules	Style rules, output-format conventions, refusal rules
C3	User facts	Stable user facts (bundle) or overridable per-call facts
C4	Short-term state	Session memory, in-conversation working state
C5	Long-term memory	Vector stores, knowledge bases, persistent recall
C6	Tools	HTTP APIs, SDKs, MCP tools, abstract capabilities
C7	Credentials	API keys, OAuth tokens, secrets
C8	Runtime environment	Model id, backend, packages, loop budgets
C9	Scheduling / triggers	Cron, webhooks, event handlers, polling loops

A.3 Mapping to MELLEA Primitives

Table 3 shows how each specification content type maps to a MELLEA construct.

Table 3: Mapping from specification content to pipeline constructs.

Spec Content	Pipeline Construct
Procedural instructions	Pipeline phases (sequential function calls)
Extraction tasks	<code>@generative</code> slots; two-step pattern for complex schemas
Classification logic	<code>@generative</code> $\rightarrow$ <code>Literal[...]</code> for control flow routing
Generation tasks	<code>m.instruct(format=Schema)</code> with <code>requirements</code> and <code>RepairTemplateStrategy</code>
Domain validation	Two-step: <code>@generative</code> <code>extract</code> $\rightarrow$ <code>m.instruct()</code> with <code>grounding_context</code>
Output constraints	<code>Requirement</code> with <code>validation_fn</code> (structural) or LLM-as-judge (semantic)
Negative constraints	<code>check()</code> validators (suppressed from generation prompt)
Domain knowledge	Config constants + <code>grounding_context</code> dicts
Deterministic logic	Plain Python functions (no LLM calls)
Tool invocations	Deterministic templates with domain allowlists
Persona / style	<code>prefix=</code> parameter; remains as prompt text

Two mapping decisions merit particular attention. First, the compiler distinguishes *output validation* (checks on the agent’s own generated text) from *domain validation* (checks the agent performs on external artifacts such as code or data). Output validation becomes `Requirement` objects with Python `validation_fn` for structural checks and LLM-as-Judge for semantic checks. Domain validation becomes the *two-step pattern*: a `@generative` slot extracts structured data from the input, then `m.instruct(format=Schema)` produces an assessment with the extracted data as `grounding_context`. This two-step pattern is the primary compilation strategy for analytical skills (Type A).

Second, schema complexity governs the choice between `@generative` slots and `m.instruct()` calls. Simple schemas (≤ 4 fields, no `Literal` constraints, no nested models) are extracted directly via `@generative`, which returns typed values but has no retry mechanism. Complex schemas use the two-step pattern with `RepairTemplateStrategy`, which retries and repairs malformed JSON up to a configurable budget (`LOOP_BUDGET`, typically 3). This distinction is necessary because small language models frequently produce truncated or malformed JSON for complex schemas, a failure mode observed across all six evaluation models.

A.4 Generated File Set

Table 4 shows the complete generated package structure.

Table 4: Generated package structure. Files marked *conditional* are only produced when the corresponding classification axis requires them.

File	Purpose	When Generated
pipeline.py	Orchestration logic	Always
schemas.py	Pydantic output models	Always
config.py	Extracted thresholds + model config	Always
requirements.py	Structural & semantic validators	Always
pyproject.toml	Package metadata for <code>pip install</code>	Always
slots.py	@ generative extraction functions	Types A, B, C
tools.py	Tool functions with allowlists	Pipeline calls tools
loader.py	Reference file reader	External dependencies
fixtures/	5–8 test fixtures (factory functions)	Always
mapping_report.md	Provenance: spec $\rightarrow$ pipeline	Always
dependencies.yaml	Machine-readable dependency manifest	Tools provide input
SETUP.md	Developer action list with commands	Tools provide input

A.5 Dependency Dispositions

Every external dependency identified during inventory (Step 1) and mapping (Step 2) is assigned one of eight *dispositions* at Step 2.5. The disposition records the explicit commitment about how the compiler should handle that dependency at code-generation time. Table 5 lists each disposition with its semantics and the dependency category that selects it as the default when no override is supplied.

Table 5: Dependency dispositions assigned at Step 2.5.

Disposition	Effect	Default for
bundle	Embed the value as a <code>Final[str]</code> constant in <code>config.py</code> .	C1 identity, C2 operating rules, C3 stable user facts, C8 runtime environment
real_impl	Generate a real implementation (HTTP call, SDK usage, etc.).	C6 tools with a concrete endpoint
stub	Generate a <code>NotImplementedError</code> placeholder for the integrator to complete.	C6 tools described abstractly
mock	Generate a fixture-backed mock for testing or demo use.	— <i>override only</i>
delegate_to_runtime	The host runtime (session, memory, scheduler) provides this.	C4 short-term state, C5 long-term memory, C9 scheduling
external_input	Supplied at invocation time as a CLI flag or environment variable.	C3 overridable user facts, C7 credentials
load_from_disk	Read from a local file at runtime (reference docs, config files).	— <i>override only</i>
remove	Source element is a cross-reference artifact; produce no code.	cross-reference elements

Three elicitation modes determine how dispositions are chosen. **auto**, the default, applies category defaults silently; if any default produces a stub or runtime delegate, Step 5 emits a recap to **stdout** so the user is aware. **ask** opens a terminal UI; the user reviews each dependency in turn and accepts the default or selects an alternative. **strict** applies category defaults and then halts before any

code is emitted if any disposition would resolve to a stub; the user must re-run with `ask` or supply a config file with explicit `real_impl` dispositions. Every amendment to a default disposition is recorded in `intermediate/element_mapping_amendments.json`, so the full audit trail is reviewable post-compile.

When detected, the compiler also generates `dependencies.yaml` (a machine-readable manifest of non-bundle dependencies), `SETUP.md` (a developer action list with copy-pasteable commands for any work the user must complete before deployment), and optionally `loader.py` (a thin file reader for `load_from_disk` dependencies).

A.6 Static Validation Lints

Step 7 runs the lints listed in Table 6. “Repairable” failures re-enter Step 4 with the lint message and a three-line context window for at most two repair rounds. “Halt” failures are not repaired because their root cause cannot be reliably patched without regenerating the whole file (architectural mistakes, security concerns). A tenth `melleafy-json-consistency` check runs alongside Tier 3 and verifies that `melleafy.json` agrees with the classification and dependency plan, surfacing any cross-artifact drift introduced by a partial regeneration.

Table 6: The nine formal lints partitioned into three tiers.

Tier	Lint	Defends	Routing
1	<code>parseable</code>	Every <code>.py</code> file parses; entry module imports cleanly without <code>ModuleNotFoundError</code>	repairable
2	<code>cross-reference</code>	Mapped target symbols exist; every external call has a dependency-plan entry; no dead slots or requirements	repairable
2	<code>session-boundary</code>	At most one <code>BaseModel</code> format type per MELLEA session (schema-priming discipline)	halt
2	<code>variable-safety</code>	No uninitialised names in <code>except/finally</code> ; no shadowed Python builtins	repairable
2	<code>import-side-effects</code>	No network calls or <code>load_dotenv()</code> at module import time	repairable
2	<code>import-soundness</code>	Every <code>from mellea.X import Y</code> resolves to a real module path	repairable
2	<code>stdlib-arity</code>	Every call to a known MELLEA <code>stdlib</code> function matches its introspected signature	repairable
2	<code>known-behaviours</code>	Mechanical checks for catalogued small-model failure patterns (validator signatures, return-type discipline, reserved parameter names, deprecated backends)	repairable
3	<code>category-specific</code>	Entropy and credential scans on persona text and config constants; MCP tool-name conventions	halt

B. Instrumentation Details

This appendix provides the full technical details of the policy-driven guardrail instrumentation summarised in Section 3.2.

B.1 Policy Manifest Generation

The manifest is generated via the following procedure:

1. **Guardian risk identification.** The skill's use-case description (composed from the `SKILL.md` frontmatter, tool sensitivity classification, and specification body) is submitted to AI Atlas Nexus:

```
nexus.identify_risks_from_usecases(
    [use_case], engine,
    taxonomy="ibm-granite-guardian",
    zero_shot_only=True)
```

This returns a list of applicable risks from the Guardian taxonomy, each with a name, description, concern text, and (critically) an optional `tag` field that identifies Guardian-native risk dimensions.

2. **Native vs. custom classification.** Each identified risk is classified into one of two tiers based on the presence of the Nexus `tag` field:

Native risks have a `tag` (e.g., "harm", "jailbreak", "social_bias"). The bare tag is sent to Guardian as the system prompt, activating Guardian's calibrated assessment path for that dimension.

Custom criteria lack a `tag`. The risk description and concern text are concatenated and sent as free-form criteria.

3. **Governance taxonomy mapping.** For each configured governance taxonomy (by default NIST AI RMF and Credo UCF), the use case is submitted to Nexus again. The returned governance risks are expanded via `nexus.get_related_actions(risk)` to collect actionable requirements (e.g., NIST actions `GV-1.2-001`, Credo controls). Cross-taxonomy links are followed: Guardian risks are mapped to related Credo UCF actions even when the Credo taxonomy is not directly queried.

The resulting `PolicyManifest` dataclass captures the use case, the identified `NexusRisk` objects (each with `guardian_prompt`, `is_native`, and taxonomy metadata), the collected `GovernanceAction` objects (each with source taxonomy, NIST category, and the linking risk), and generation metadata (timestamp, model used).

B.2 Guardian Hook Architecture

The manifest drives the creation of MELLEA hook plugins via a factory method:

```
plugin = GuardianAuditPlugin.from_manifest(manifest, enforce=False)
```

Both plugins register on three hook points: `GENERATION_PRE_CALL` (checks the input prompt before the LLM is called), `GENERATION_POST_CALL` (checks the input-output pair after generation), and `TOOL_PRE_INVOKE/TOOL_POST_INVOKE` (for LLM-directed tool calls). Each hook iterates over all risks in the manifest, calling Guardian once per risk:

```
messages = [
    {"role": "system", "content": risk_prompt},
    {"role": "user", "content": user_text},
    {"role": "assistant", "content": output} # post-call only
]
```

Guardian responses are parsed for `<score>yes</score>` or `<score>no</score>` tags, producing a `GuardianVerdict` with fields `risk`, `label` ("Yes", "No", or "Failed"), `raw_output`, and `timestamp`. Verdicts are stashed in MELLEA's `user_metadata` dictionary for downstream consumption by the audit trail plugin.

B.3 Audit Trail

An `AuditTrailPlugin` (running after Guardian in priority order) writes a JSONL log of every pipeline event. Seven hook points are instrumented: `generation_pre_call`, `generation_post_call`, `component_pre_execute`, `component_post_success`, `component_post_error`, `validation_post_check`, and `tool_pre_invoke/tool_post_invoke`. Each entry includes a UTC timestamp, session and request identifiers, and an optional `policy_id` linking back to the manifest. Post-generation entries include Guardian verdicts and latency measurements.

B.4 Tool Sensitivity Classification

Before policy generation, the skill’s declared tools are classified into sensitivity tiers using a static lookup table.

Table 7: Tool sensitivity classification. The highest tier across all declared tools determines the skill’s overall sensitivity.

Tier	Operations	Example Tools
Non-sensitive	read	Read, Grep, Glob
Sensitive	read, write, execute	Bash, Edit, Write
Restricted	auth_credentials	op, credential stores

Body-level pattern matching escalates the tier when the specification text contains keywords indicating capabilities not visible in the tool list (e.g., `api_key`, `secret`, `delete`, `deploy`). The resulting sensitivity feeds into the use-case description submitted to Nexus, influencing which risks are identified.